INDICE ESTRUCTURA VISTAS

01 ESTRUCTURA PRODUCTOS.....1
Los 15 productos propuestos..... 1
2. Qué hace cada vista.....3

—----------VISTA 1 EL CARRITO—----------..... 6
1. Funcionamiento del aside..... 6
2. Validación: aquí está la clave..... 7
3. Los dos botones inferiores..... 7
Mi recomendación técnica.....7
3. La lógica de validación y envío..... 8
4. Después de validar: dos caminos..... 9

----------VISTA 2 LISTAR EL CATÁLOGO—----------.....10
1. ¿Cómo funciona el filtrado progresivo?.....10
2. Vista de creación..... 11
3. Una ventaja técnica..... 11

----------VISTA 3 CREAR PRODUCTO—----------..... 13

----------VISTA 4 EL CREAR PROYECTO —----------.....14
Cómo funcionaría..... 14
Distribución de la vista..... 17

----------VISTA 5 LISTAR PROYECTOS —----------..... 18

----------VISTA 6 LISTAR ARQUITECTURAS—----------..... 20

----------VISTA 7 LISTAR PRESUPUESTOS----------.....21
Flujo de funcionamiento..... 22

OJO EL FLUJO DE CARRITO , ETC VA A MYSQL Y SE MANTIENE.

01 ESTRUCTURA PRODUCTOS

Los 15 productos propuestos

Tipo	Producto AWS	Producto Azure
Computación	EC2	Virtual Machines
Computación	Lightsail	—
Almacenamiento	S3	Blob Storage
Almacenamiento	—	Managed Disks
Bases de datos	RDS	Azure SQL Database
Bases de datos	DynamoDB	—
Redes	VPC	Virtual Network
Seguridad	IAM	Key Vault
Monitorización	CloudWatch	Azure Monitor

MI VERSIÓN SIMPLIFICADA: Un Proyecto es un objeto con dos atributos; Descripción - string. Presupuesto - number y un array de requisitos que son strings. Y luego si tiene o no una arquitectura asociada o varias.

1. Estructura de las seis vistas



2. Qué hace cada vista

Vista	Funcionalidades
1. Índice	Accesos a las otras cinco vistas y resumen general.
2. Catálogo	Crear, listar, consultar, editar y desactivar productos.
3. Proyectos	Crear, listar, consultar, editar y eliminar proyectos. Gestionar sus requisitos.
4. Carrito	Seleccionar productos, cantidades y configuraciones. Calcular precios y aplicar reglas.
5. Arquitecturas	Consultar arquitecturas guardadas, sus productos y el proyecto al que pertenecen.
6. Presupuestos	Consultar presupuestos emitidos, sus importes, líneas, fechas y proyecto o arquitectura asociados.

Una precisión importante: el carrito no debería ser solamente un array de objetos JavaScript. Podemos utilizar un `select` con `option value` para seleccionar productos y un array de objetos para representar la selección en la interfaz, pero Laravel debe validar y guardar las operaciones en el servidor y en la base de datos.

Mantenemos los **15 productos, 6 tipos y 2 proveedores**, pero añadimos una propuesta de distribución regional.

1. Los 15 productos propuestos

Tipo	Producto AWS	Producto Azure
Computación	EC2	Virtual Machines
Computación	Lightsail	—
Almacenamiento	S3	Blob Storage
Almacenamiento	—	Managed Disks
Bases de datos	RDS	Azure SQL Database
Bases de datos	DynamoDB	—
Redes	VPC	Virtual Network
Seguridad	IAM	Key Vault
Monitorización	CloudWatch	Azure Monitor

2. Las cinco regiones propuestas

Proveedor	Región	Código
AWS	Europa (Irlanda)	eu-west-1
AWS	Europa (Frankfurt)	eu-central-1
AWS	Europa (Londres)	eu-west-2
Azure	West Europe	westeurope
Azure	North Europe	northeurope

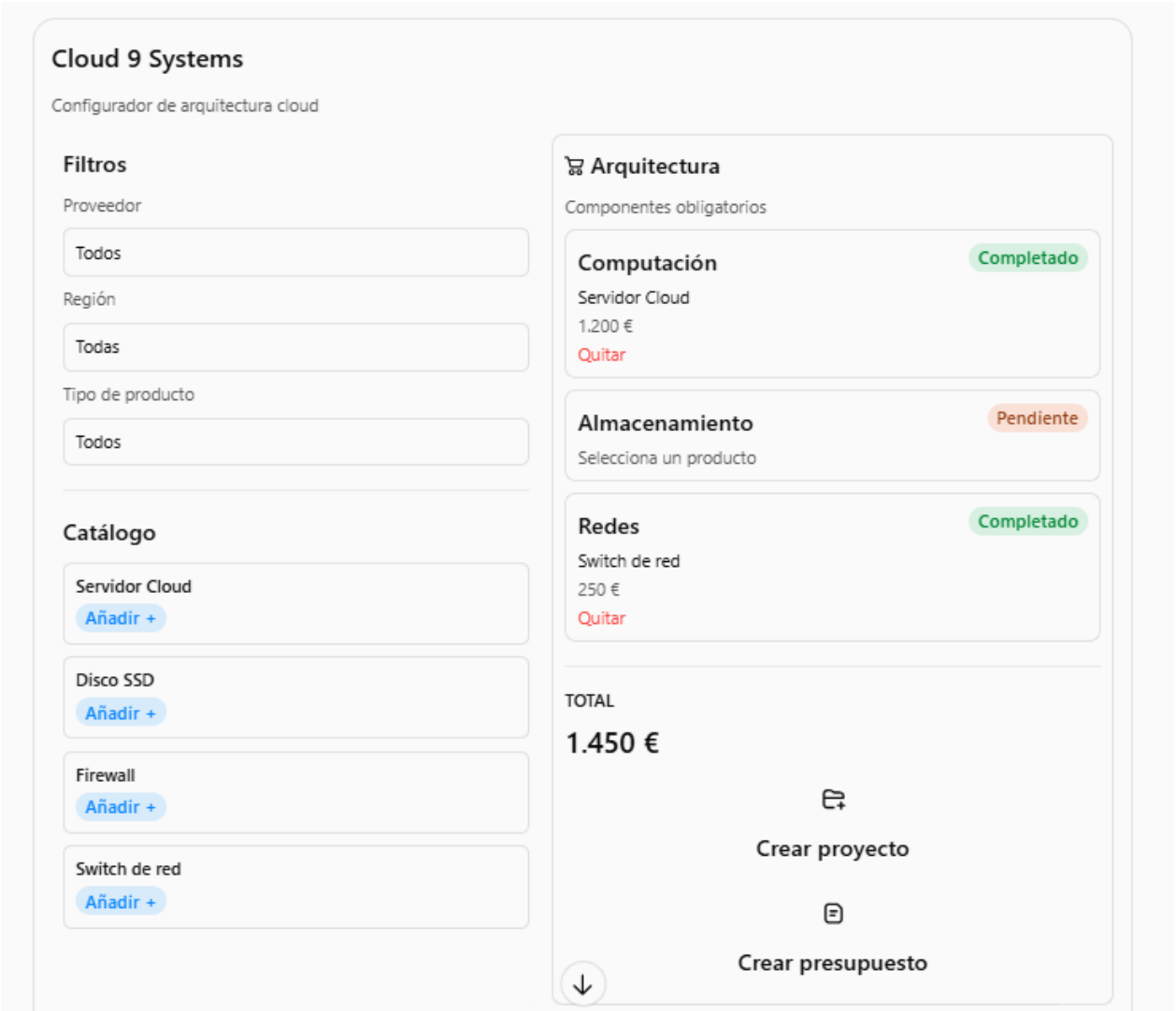
3. Disponibilidad regional de los productos

Esta sería nuestra matriz inicial de datos de prueba. Los precios se introducirán por separado en la relac
producto-región.

S = Disponible · N = No disponible. Distribución didáctica propuesta, no inventario verificado de disponibilidad comercial.

Producto	Irlanda	Frankfurt	Londres	West Europe	North Eurc
AWS EC2	S	S	S	—	—
AWS Lightsail	S	N	S	—	—
AWS S3	S	S	S	—	—
Azure Blob	—	—	—	S	S
Azure Managed Disks	—	—	—	S	N
AWS RDS	S	S	N	—	—
Azure SQL Database	—	—	—	S	S
AWS DynamoDB	S	S	S	—	—
AWS VPC	S	S	S	—	—
Azure Virtual Network	—	—	—	S	S
AWS IAM	S	S	S	—	—
Azure Key Vault	—	—	—	S	S
AWS CloudWatch	S	N	S	—	—
Azure Monitor	—	—	—	S	S

—----------VISTA 1 EL CARRITO—----------



1. Funcionamiento del aside

Categoría	Estado inicial	Al seleccionar
Computación	Pendiente	Verde
Almacenamiento	Pendiente	Verde
Redes	Pendiente	Verde

- Los tres bloques aparecen desde el principio.
- Cuando se añade un producto de una categoría, se completa su bloque.
- Se muestra el producto, precio y opción de quitar.
- Al quitarlo, vuelve a quedar pendiente.

2. Validación: aquí está la clave

Quiero destacar lo que has propuesto porque es importante para JavaScript.

Tendremos dos momentos de validación:

Onblur

- El usuario abandona un campo.
- Si está vacío o es incorrecto, borde rojo y mensaje en un `span`.
- Si es correcto, desaparece el error.

Submit

- Se comprueban todos los bloques obligatorios.
- Si falta almacenamiento, redes o computación, no se permite continuar.
- Se muestran los errores correspondientes.

Por ejemplo:

Almacenamiento

Debes seleccionar un producto de almacenamiento.

3. Los dos botones inferiores

- Crear proyecto: guardar la arquitectura y asociarla a un proyecto.
- Crear presupuesto: generar el presupuesto identificado por su nombre.

Los dos exigirán que la composición esté completa y validada.

Mi recomendación técnica

Mantendría una estructura de datos independiente para cada categoría y un estado de selección que podamos consultar para validar el carrito completo.

Y una precisión: si el administrador quita el único producto de una categoría, esa categoría vuelve a estar pendiente.

CON EL ASIDE LATERAL DE LA ELECCIÓN DE PRODUCTOS COMO HACER EL ASIDE EN HTML /CCSSS

2. ¿Es complicado hacerlo en CSS?

No. La clave está en utilizar **CSS Grid o Flexbox**.

Por ejemplo, la estructura HTML sería:

HTML

```html
<main class="configurador">

    <section class="seleccion-productos">
        <!-- Proveedor y región -->
        <!-- Productos por categoría -->
    </section>

    <aside id="carrito">
        <!-- Lista dinámica de productos -->
        <!-- Total -->
    </aside>

</main>
```

Y el CSS básico:

CSS

```css
.configurador {
    display: grid;
    grid-template-columns: 2fr 1fr;
    gap: 25px;
    align-items: start;
}

#carrito {
    position: sticky;
    top: 20px;
}
```

Ventajas:

- El usuario conoce todo el catálogo.
- Puede ver qué servicios no están disponibles en su región.
- No puede seleccionar productos incompatibles.
- No necesitamos otro filtro por categorías.
- Cumplimos expresamente la demostración de disponibilidad regional.

3. La lógica de validación y envío

Aquí también estoy de acuerdo contigo, pero hay un pequeño matiz técnico: `preventDefault()` pertenece a JavaScript, no a PHP.

El flujo correcto sería:



En Laravel recibiremos el formulario mediante una ruta POST y ejecutaremos la validación en el controlador o mediante un Form Request.

No necesitamos `preventDefault()` si hacemos un envío tradicional de formulario. Laravel recibe el POST directamente.

4. Después de validar: dos caminos

A. Presupuesto

1. Seleccionar tipo de pago.
2. Tarjeta o fraccionado.
3. Guardar el presupuesto.

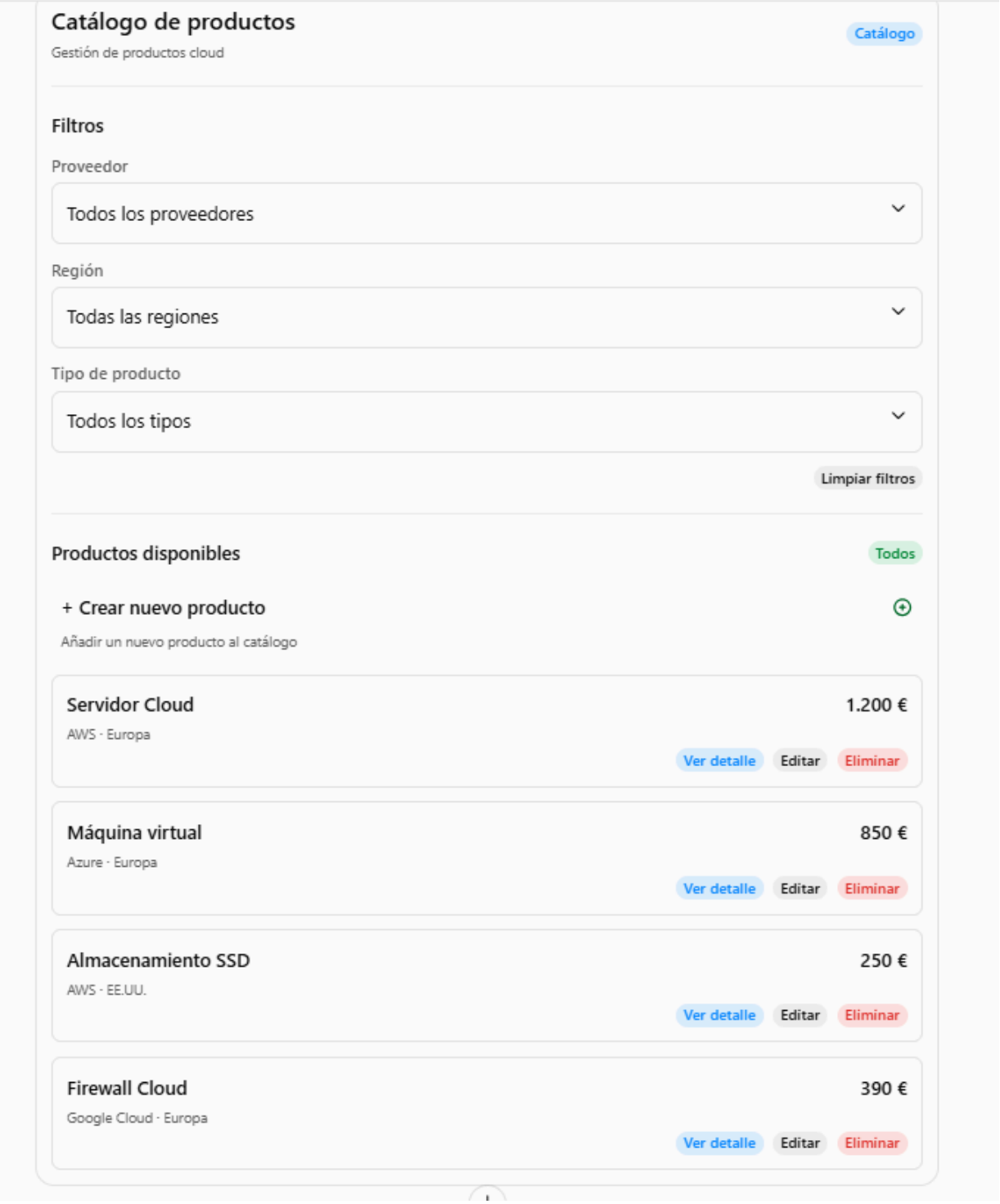
B. Arquitectura

1. Seleccionar proyecto existente.
2. Comprobar que se ha seleccionado uno válido.
3. Si no existen proyectos, impedir la creación.
4. Asociar la arquitectura al proyecto.
5. Guardar la arquitectura y sus productos.
6. Asociarlo al proyecto si corresponde según los requisitos.

Una cosa que quiero dejar clara: la validación definitiva debe hacerse en el servidor, aunque visualmente demos avisos antes.

Yo elegiría siempre visible (`position: sticky`). Así el usuario puede añadir diez productos sin perder de vista su selección.

----------VISTA 2 LISTAR EL CATÁLOGO—----------



1. ¿Cómo funciona el filtrado progresivo?

Aquí está la clave:

1. Al entrar aparecen todos los productos.
2. Seleccionas un proveedor y se actualizan los productos.
3. El SELECT de región muestra las regiones disponibles para ese proveedor.
4. Seleccionas una región y se actualizan los tipos de producto disponibles.
5. Seleccionas el tipo y obtienes el resultado definitivo.

Por ejemplo:

`AWS → Europa → Computación`

Los tres filtros se acumulan. Si cambias el proveedor, se reinician los filtros dependientes que ya no sean válidos.

2. Vista de creación

Aquí mantenemos la otra propuesta que te gustaba. No vamos a cambiarla por este nuevo diseño de listado. Conservamos aquel planteamiento visual y su formulario.

Decisión tomada

3. Una ventaja técnica

En Laravel podremos tener una única consulta de productos que vaya incorporando condiciones según los SELECT seleccionados.

Y en JavaScript podemos gestionar los cambios de los SELECT para actualizar los resultados sin duplicar código.

Mi veredicto: listado limpio, tres filtros encadenados y creación independiente.

Esta será nuestra filosofía para el catálogo de Cloud 9 Systems. No mezclemos la gestión de productos con el configurador de arquitecturas.

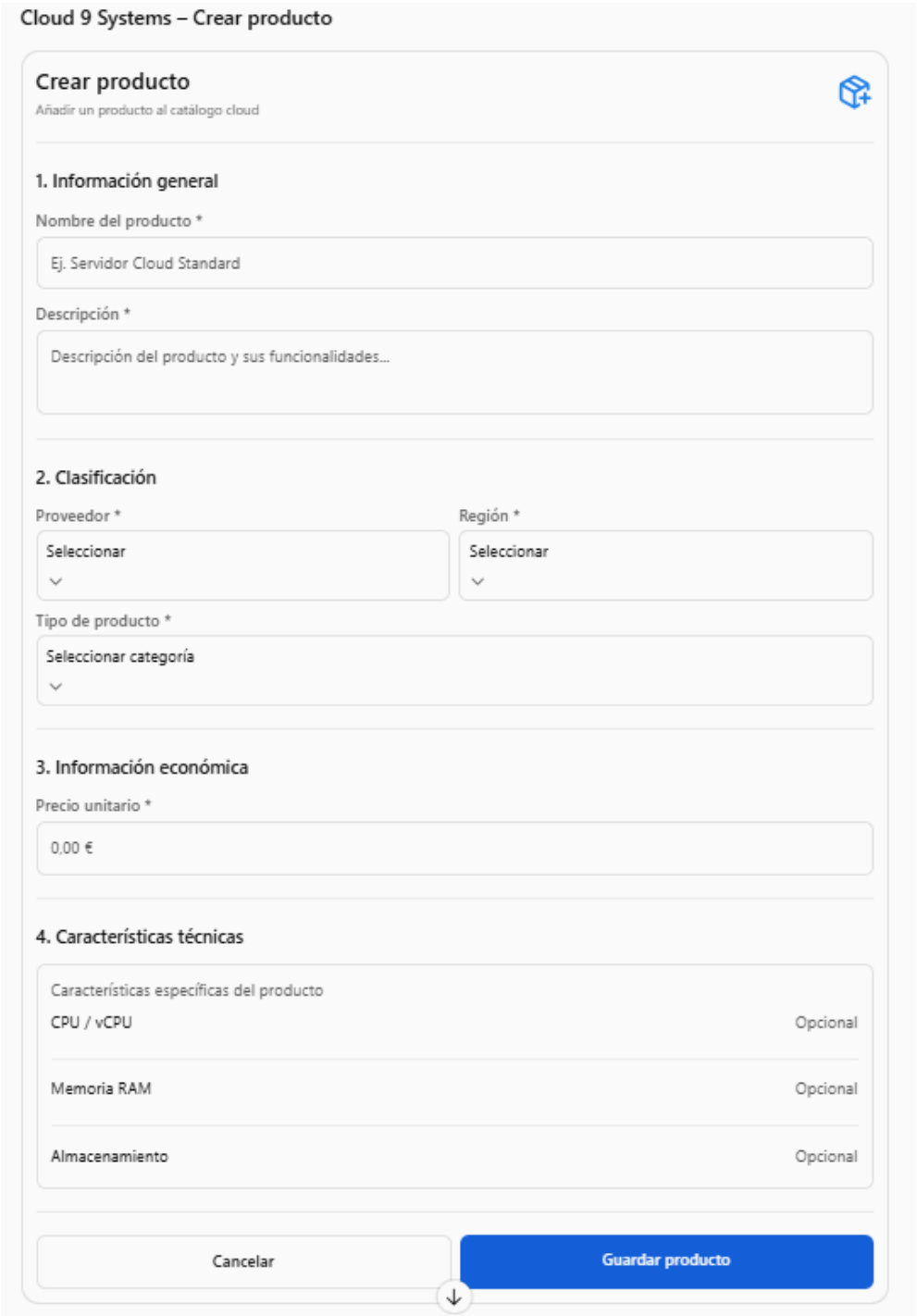
I:

Operación	Método HTTP
Listar productos	GET
Filtrar productos	GET
Crear producto	POST
Editar producto	PATCH
Eliminar/desactivar	DELETE

.

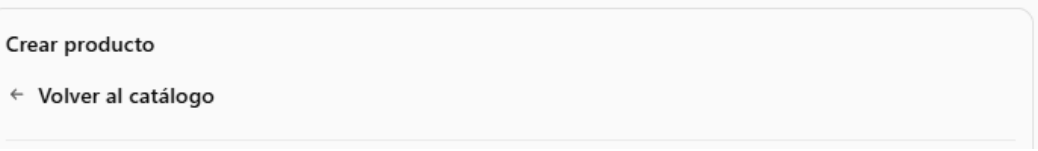
Y una decisión importante: proveedor, región y tipo de producto deben utilizar los mismos registros que los filtros del catálogo. Así evitamos inconsistencias

----------VISTA 3 CREAR PRODUCTO—----------

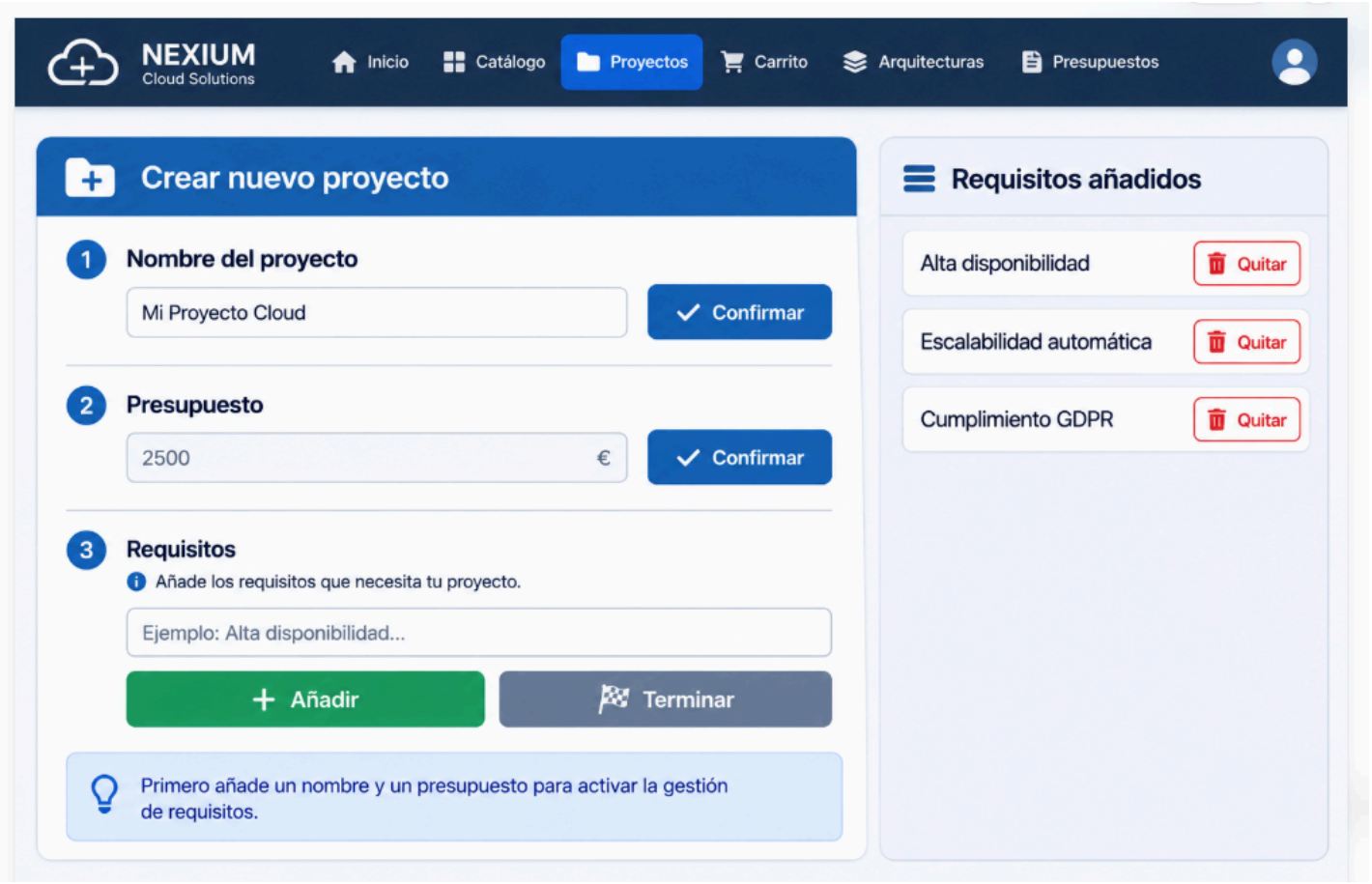


2. Botones de retorno

Esto es importantísimo. No queremos que el usuario se quede atrapado en un formulario.



----------VISTA 4 EL CREAR PROYECTO —----------



Cómo funcionaría

1. **El usuario introduce el nombre y pulsa Confirmar.**
2. **El nombre queda bloqueado.**
3. **Introduce el presupuesto y pulsa Confirmar.**
4. **El presupuesto queda bloqueado.**
5. **Se habilita la creación de requisitos.**
6. **Escribe un requisito y pulsa Añadir.**
7. **El requisito se incorpora al listado lateral.**
8. **Puede seguir añadiendo requisitos.**
9. **Pulsa Terminar y finaliza la creación del proyecto.**

Añadir y Terminar: son dos acciones distintas.

- **Añadir: incorpora un requisito a la colección.**
- **Terminar: guarda o finaliza el proyecto con sus requisitos.**

Vista Blade — Aside de requisitos

```
<aside class="aside-requisitos">

    <h3>Requisitos añadidos</h3>

    <div class="lista-requisitos">

        @forelse($requisitos as $index => $requisito)

            <div class="requisito-item">

                <span class="requisito-texto">
                    {{ $requisito }}
                </span>

                <form method="POST"
                      action="{{ route('requisitos.destroy', $index) }}">

                    @csrf
                    @method('DELETE')

                    <button type="submit" class="btn-quitar">
                        Quitar
                    </button>

                </form>

            </div>

        @empty

            <p class="sin-requisitos">
                Todavía no hay requisitos añadidos.
            </p>

        @endforelse

    </div>

</aside>
```

CSS DE REQUISITOS

```
.aside-requisitos {
    width: 300px;
    min-width: 260px;
    padding: 20px;
    background-color: #f5f6f8;
    border: 1px solid #ddd;
```

```
    border-radius: 10px;
    position: sticky;
    top: 20px;
    height: fit-content;
}

.aside-requisitos h3 {
    margin-top: 0;
    margin-bottom: 20px;
}

.lista-requisitos {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.requisito-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 10px;
    padding: 12px;
    background-color: white;
    border: 1px solid #ddd;
    border-radius: 8px;
}

.requisito-texto {
    overflow-wrap: anywhere;
    flex: 1;
}

.btn-quitar {
    border: none;
    background-color: #dc3545;
    color: white;
    padding: 7px 10px;
    border-radius: 5px;
    cursor: pointer;
}

.btn-quitar:hover {
    background-color: #b02a37;
}

.sin-requisitos {
    color: #777;
```

```
    font-size: 14px;
}
```

Distribución de la vista

La estructura general quedaría así:

CONTENEDOR PRINCIPAL

```
│
├── SECTION (Crear proyecto)
│    ├── Nombre
│    ├── Presupuesto
│    └── Añadir requisitos
│
└── ASIDE (Requisitos añadidos)
     ├── Requisito 1    [Quitar]
     ├── Requisito 2    [Quitar]
     └── Requisito 3    [Quitar]
```

2. Botones de retorno

Esto es importantísimo. No queremos que el usuario se quede atrapado en un formulario.

Crear producto

← Volver al catálogo

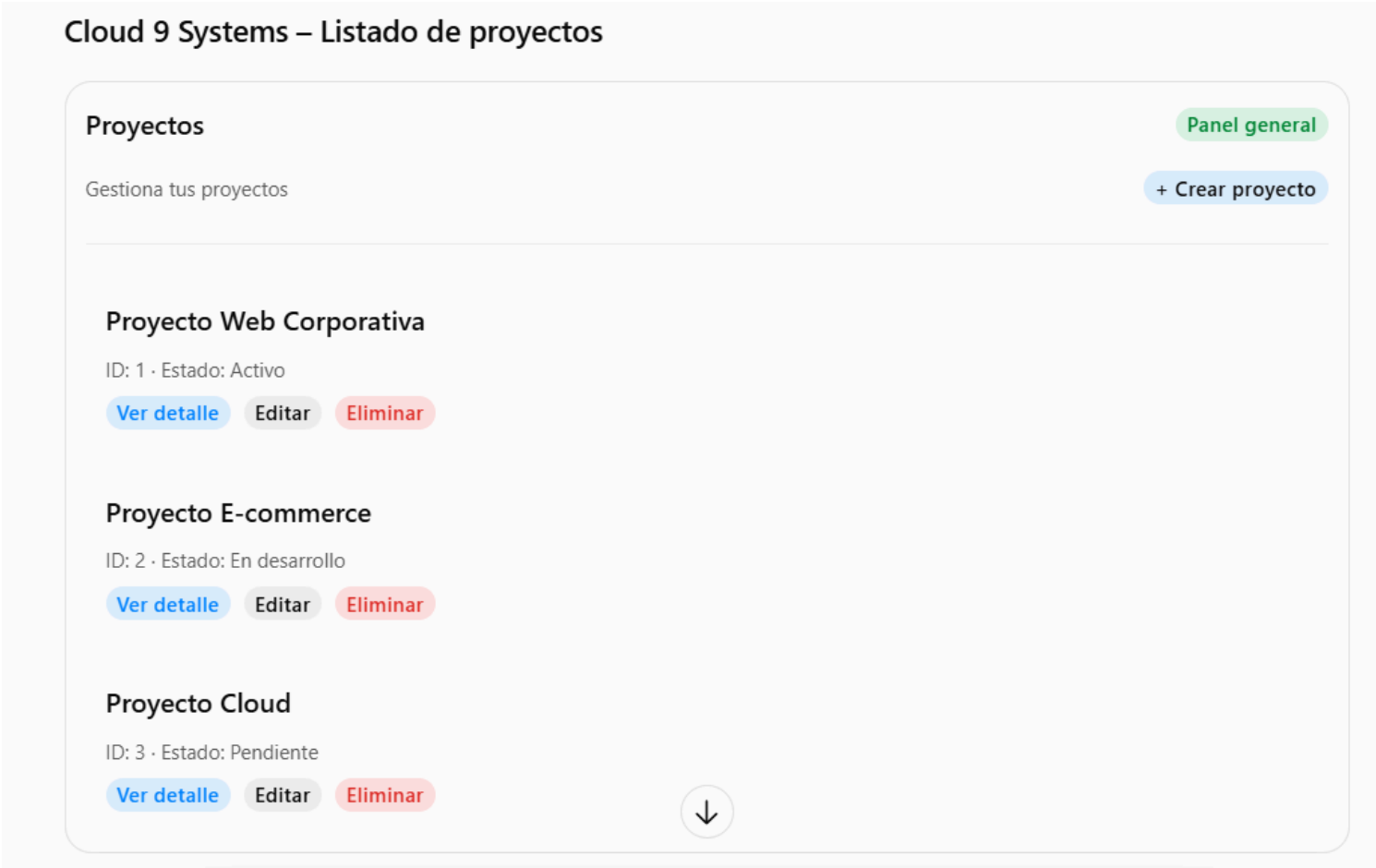
Crear proyecto

← Volver a proyectos

El retorno estará claramente visible, preferiblemente en la parte superior del contenido.

Y también mantendremos los botones de cancelar y guardar en los formularios.

----------VISTA 5 LISTAR PROYECTOS —----------



Y la estructura funcional sería:

Elemento	Acción Laravel
Crear proyecto	projects.create
Ver detalle	projects.show
Editar	projects.edit
Eliminar	projects.destroy

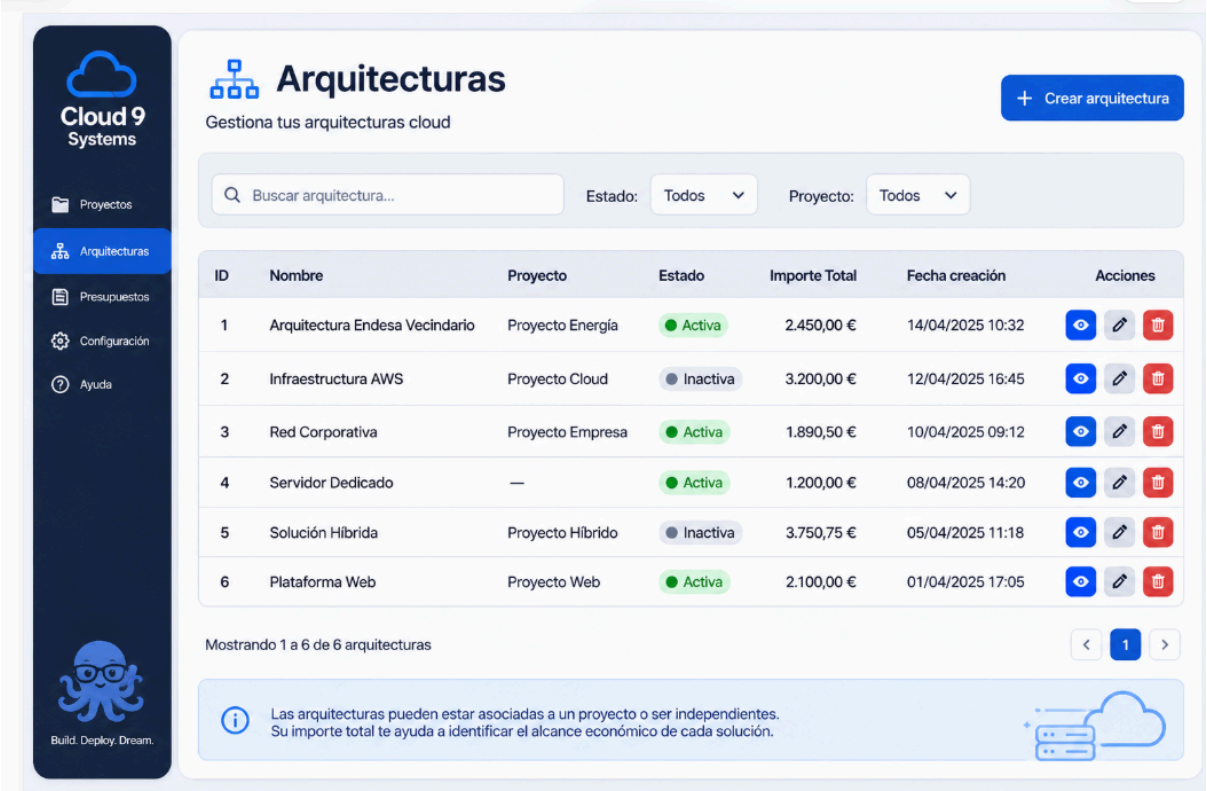
Cómo lo desarrollaría

1. Ruta GET: acceder al listado de proyectos.
2. Controller: método index() para recuperar los proyectos de la base de datos.
3. Modelo: utilizar Project para obtener los registros.
4. Vista Blade: recorrer los proyectos con @foreach.

5. Botones dinámicos: cada fila tendrá sus tres acciones asociadas al ID único del proyecto.
6. Eliminar: formulario con método DELETE, protección CSRF y confirmación.

Una cosa importante: los botones de editar, eliminar y ver detalle estarán dentro de cada iteración del foreach. Así cada uno trabajará con el proyecto correspondiente, sin confundir registros.

----------VISTA 6 LISTAR ARQUITECTURAS—----------



Campo	Descripción
ID	**Identificador único**
Nombre	**Máximo 20 caracteres**
Proyecto	**ID o nombre del proyecto asociado, o vacío**
Estado	**Activa / Inactiva**
Importe total	**Coste total de la arquitectura**
Fecha creación	**Fecha y hora de creación**

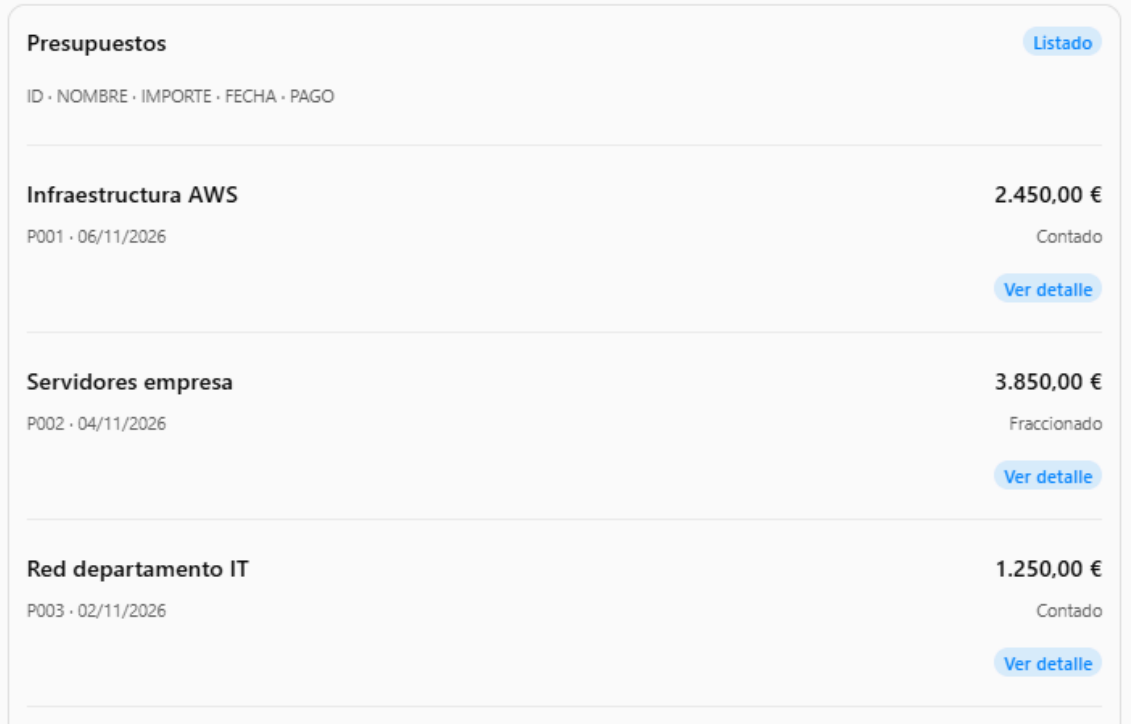
1. Los 20 caracteres me parecen perfectos. «Arquitectura Endesa Vecindario» podría abreviarse a «Endesa Vecindario», por ejemplo.
2. El importe total debería calcularse a partir de los productos incluidos en la arquitectura, no introducirse manualmente.

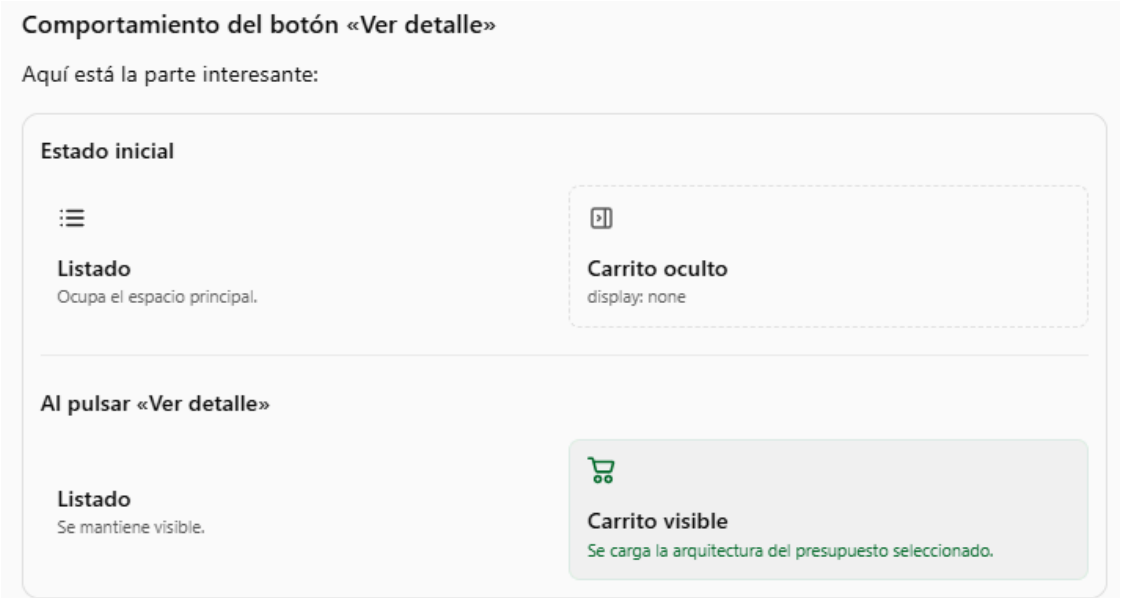
----------VISTA 7 LISTAR PRESUPUESTOS----------

Aquí vamos a reutilizar una solución que ya tenemos diseñada: **el carrito de la vista de configuración**, pero ahora funcionará como panel de consulta de un presupuesto guardado.

Vista: Listado de presupuestos

La estructura sería sencilla, siguiendo el estilo de los listados de proyectos y arquitecturas.





El panel derecho mostraría:

- Productos de Computación.
- Productos de Almacenamiento.
- Productos de Redes.
- Cantidades y precios.
- Importe total.
- Tipo de pago.
- Botón para cerrar el detalle.

Y un detalle técnico importante: al pulsar sobre un presupuesto, no vamos a reutilizar un carrito vacío ni el carrito actual del configurador. Vamos a recuperar los productos asociados al ID del presupuesto seleccionado y pintarlos en el panel.

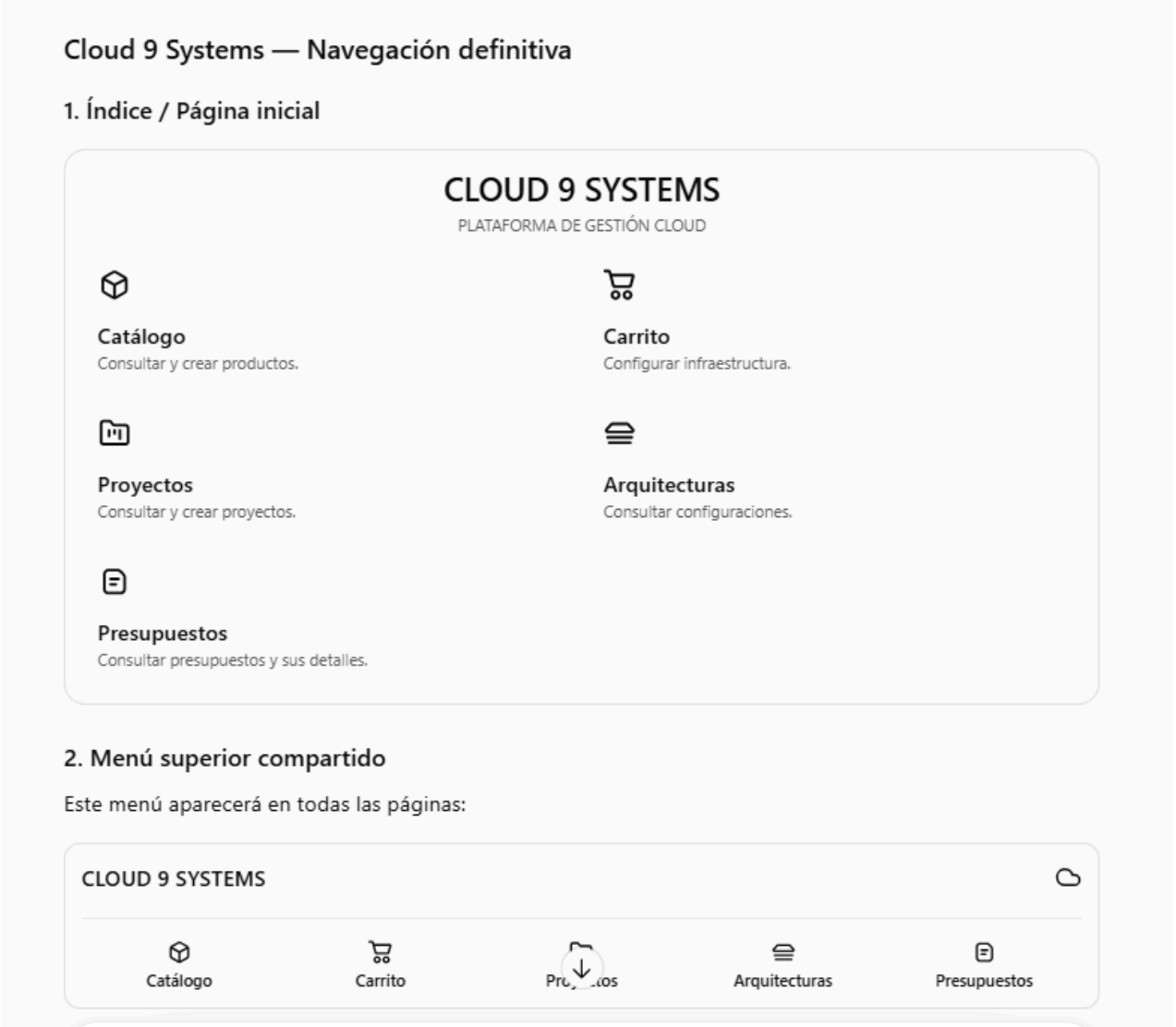
Así podemos consultar diferentes presupuestos sin modificar su contenido.

Flujo de funcionamiento

1. Entramos en budgets.index.
2. Laravel obtiene todos los presupuestos.
3. Pulsamos «Ver detalle» en uno de ellos.
4. Se recupera su información y sus productos.
5. Se muestra el aside derecho.
6. Podemos cerrar el panel y seleccionar otro presupuesto.

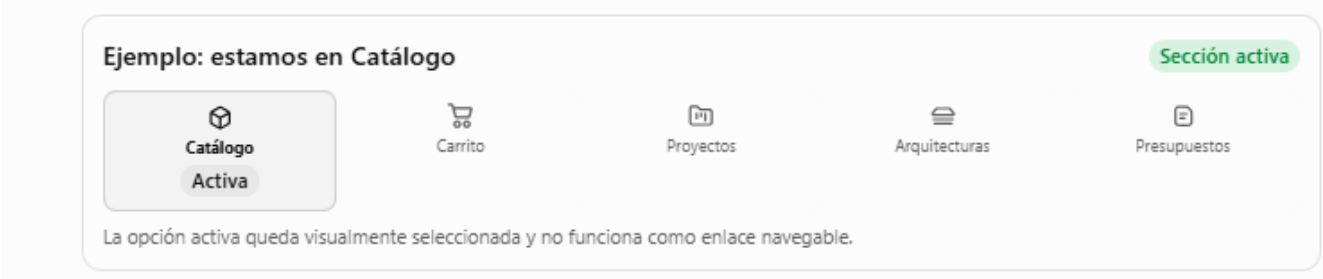
Mi propuesta: que el detalle sea un panel lateral desplegable, sin abandonar el listado. Así mantenemos el contexto y podemos comparar presupuestos rápidamente.

----------VISTA 8 ÍNDICE—----------



1. Menú superior inteligente

Tendremos cinco opciones:

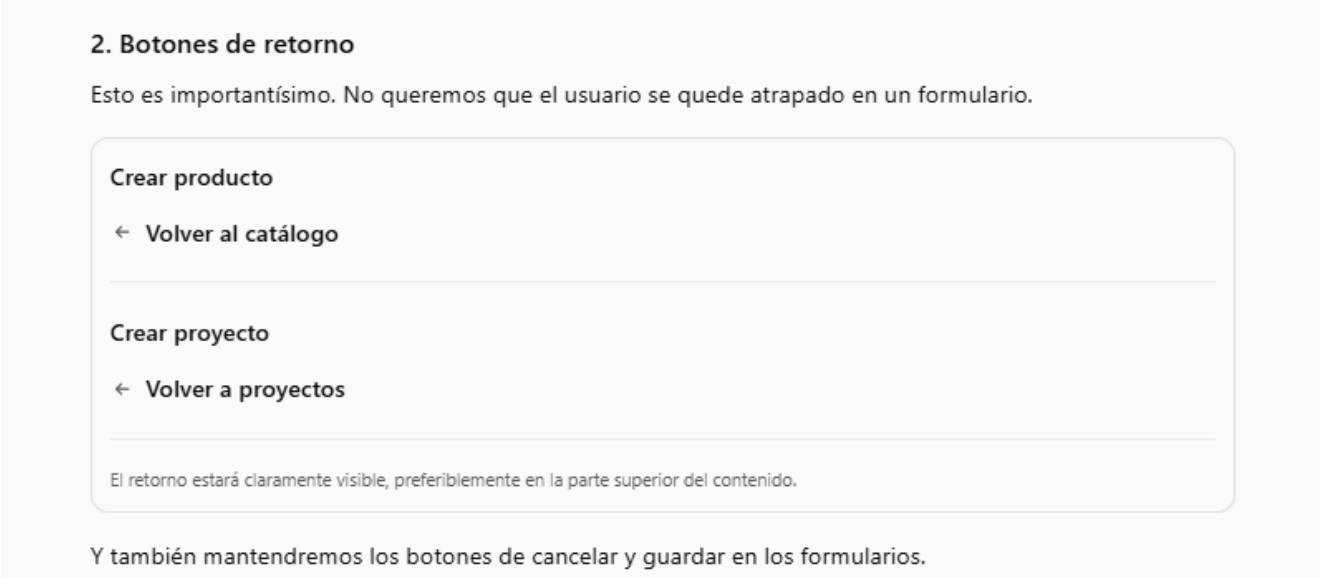


Tendremos cinco opciones:

- Catálogo
- Carrito
- Proyectos
- Arquitecturas
- Presupuestos

Esto tiene dos ventajas:

- El usuario sabe siempre dónde está.
- Evitamos recargas y navegación innecesarias hacia la misma página.



3. Navegación interna

Sección	Acciones disponibles
Catálogo	Listar productos, crear producto, editar y ver detalle
Carrito	Seleccionar productos y configurar arquitectura

Proyectos	Listar proyectos, crear proyecto, editar y ver detalle
Arquitecturas	Listar, editar y ver detalle
Presupuestos	Listar y ver detalle

4. Una regla de diseño que fijamos

- El menú superior será compartido mediante un layout común de Laravel Blade.
- El índice tendrá las mismas cinco opciones que el menú.
- Crear producto se abrirá desde Catálogo.
- Crear proyecto se abrirá desde Proyectos.
- La opción activa del menú tendrá un indicador visual.
- Las páginas conservarán el mismo encabezado y estructura general.

Estructura de navegación acordada

3. Criterios de usabilidad que fijamos

Elemento	Decisión
Buscador de productos	No implementar por ahora
Requisitos técnicos	Excluidos por acuerdo
Filtros del catálogo	Selectores sencillos
Navegación	Cinco secciones

Sección activa	Visualmente seleccionada y deshabilitada
Formularios	Botón de retorno visible
Acciones	Asociadas al ID correspondiente
Validación	Mensajes claros y campos identificados
Diseño	Consistente en todas las vistas

4. Nuestra filosofía de desarrollo

Quiero que nos quedemos con esta regla:

Cada elemento de la interfaz debe tener una función clara.

Si un botón no aporta nada, fuera. Si un campo confunde, lo simplificamos. Si una funcionalidad no podemos mantenerla correctamente, no la incorporamos por decoración.

Y hay algo que me gusta especialmente de cómo estamos trabajando: no estamos pensando solamente en cómo programar una vista, sino en cómo va a entenderla la persona que la utilice.

Eso es diseñar software, no simplemente escribir HTML, CSS y PHP.